

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG  
KHOA CÔNG NGHỆ THÔNG TIN 1**



**BÁO CÁO KỸ THUẬT THỰC HIỆN CHỨC NĂNG TRONG BTL  
ỨNG DỤNG ĐẶT XE GHÉP CHUYẾN  
RIDE UP**

|                        |                     |
|------------------------|---------------------|
| <b>Nhóm</b>            | <b>: 7</b>          |
| <b>Nhóm BTL</b>        | <b>: 4</b>          |
| <b>Thành viên nhóm</b> |                     |
| <b>Họ và tên</b>       | <b>Mã sinh viên</b> |
| Phạm Quang Huy         | : B22DCCN394        |
| Nguyễn Xuân Hưng       | : B22DCCN418        |
| Vũ Sỹ Ngọc Hiếu        | : B22DCCN322        |
| Khúc Văn Tuấn          | : B22DCCN754        |
| Nguyễn Vũ Tuấn Khôi    | : B22DCCN466        |

**Thành viên thực hiện báo cáo: Khúc Văn Tuấn**

*Hà Nội – 2026*

**PHẦN I: TÀI LIỆU KỸ THUẬT**

## 1. Danh Sách Chức Năng Được Phân Công

| STT | Chức năng                     | Mô tả                                                                                                                      | Vai trò người dùng |
|-----|-------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------|
| 1   | Đăng ký tài khoản             | Người dùng tạo tài khoản mới với vai trò CUSTOMER hoặc DRIVER. Mật khẩu được mã hóa BCrypt.                                | Khách              |
| 2   | Đăng nhập                     | Xác thực bằng email/mật khẩu, trả về JWT Access Token và Refresh Token. Access token được lưu Redis để kiểm soát vòng đời. | Tất cả             |
| 3   | Cập nhật thông tin người dùng | Cập nhật họ tên, số điện thoại, ngày sinh, giới tính, ảnh đại diện (upload file multipart lên Supabase Storage).           | Đã đăng nhập       |
| 4   | Đánh giá chuyến đi            | Hành khách đánh giá chuyến đi đã hoàn thành (1–5 sao + nhận xét). Điểm rating trung bình của tài xế được tự động cập nhật. | CUSTOMER           |

## 2. Kiến Trúc Chi Tiết Hệ Thống

### 2.1. Tổng quan kiến trúc

RideUp áp dụng kiến trúc Client–Server phân lớp (Layered Architecture) kết hợp với bộ nhớ đệm Redis và lưu trữ file trên Supabase Storage. Frontend là ứng dụng React Native (Expo), giao tiếp với backend Spring Boot qua REST API. Xác thực sử dụng JWT stateless, token được xác minh qua Redis để hỗ trợ thu hồi tức thời (Revocation).

### 2.2. Sơ đồ kiến trúc các thành phần

| Tầng      | Thành phần          | Công nghệ / Mô tả                                                                                                        |
|-----------|---------------------|--------------------------------------------------------------------------------------------------------------------------|
| Frontend  | React Native (Expo) | Màn hình: LoginScreen, RegisterScreen, CustomerHomeScreen, DriverProfileScreen. Giao tiếp backend qua axios (apiClient). |
| API Layer | REST Controller     | AuthenticationController (/auth/**), UserController (/users/**),                                                         |

|                |                       |                                                                                                                                                                   |
|----------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                |                       | CustomerBookingController (/customer/**).<br>Nhận request, xác thực, chuyển tiếp sang Service.                                                                    |
| Business Logic | Service Layer         | AuthenticationService: đăng ký, đăng nhập, refresh token, OTP đổi mật khẩu.<br>UserService: lấy/cập nhật thông tin user.<br>CustomerBookingService: tạo đánh giá. |
| Data Access    | Repository (JPA)      | UserRepository, RefreshTokenRepository, BookingReviewRepository. Truy xuất MySQL qua Spring Data JPA.                                                             |
| Cache          | Redis                 | Lưu Access Token (TOKEN: {token} → userId), OTP đổi mật khẩu, link xác minh email. Hỗ trợ revocation tức thì khi logout.                                          |
| Database       | MySQL                 | Bảng: user, user_roles, refresh_token, booking, booking_review. Kết nối qua Spring Data JPA / Hibernate.                                                          |
| File Storage   | Supabase Storage      | Lưu trữ ảnh đại diện người dùng. Upload qua FileService, path lưu vào trường avatar_url trong bảng user.                                                          |
| Security       | Spring Security + JWT | SecurityConfig cấu hình OAuth2 Resource Server. CustomJwtDecoder xác minh chữ ký HS512 và kiểm tra Redis.<br>BCryptPasswordEncoder mã hóa mật khẩu.               |

## 2.3. Luồng xử lý các chức năng chính

### Luồng Đăng ký:

1. Frontend gửi POST /auth/register với {fullName, email, password, role}
2. AuthenticationController nhận, gọi AuthenticationService.registerAccount()
3. Service mã hóa mật khẩu bằng BCrypt, gán Role (CUSTOMER/DRIVER), lưu DB
4. Trả về UserResponse (không có token – cần đăng nhập riêng)

### Luồng Đăng nhập:

1. Frontend gửi POST /auth/authentication với {email, password}
2. Service kiểm tra email tồn tại, so khớp BCrypt password
3. Sinh Access Token (JWT HS512, chứa scope/role), lưu Redis:  
TOKEN: {jwt} = userId với TTL

4. Sinh Refresh Token (UUID), hash SHA-256 rồi lưu vào bảng refresh\_token
5. Trả về {token, refreshToken, user}

#### Luồng Cập nhật thông tin:

1. Frontend gửi PUT /users/me (JSON) hoặc PUT /users/me/avatar (multipart)
2. CustomJwtDecoder giải mã JWT, kiểm tra Redis còn sống
3. UserService.updateMyInfo() lấy user từ SecurityContext, cập nhật các trường không null
4. Nếu upload avatar: FileService upload lên Supabase, lưu URL vào user.avatarUrl

#### Luồng Đánh giá chuyến đi:

1. Frontend gửi POST /customer/bookings/{bookingId}/rate với {rating, comment}
2. Kiểm tra rating hợp lệ (1–5), booking thuộc user hiện tại, trạng thái COMPLETED
3. Kiểm tra chưa đánh giá (existsByBookingId)
4. Lưu BookingReview, cập nhật lại driver.driverRating = AVG(rating)

### 3. Code Đáp Ứng Chức Năng

#### 3.1. Các lớp Entity (Bảng CSDL)

##### Entity: User – Bảng user

Đại diện tài khoản người dùng. Có quan hệ 1-1 với DriverProfile và quan hệ n-n với bảng user\_roles.

| Trường (Java) | Cột DB    | Kiểu            | Mô tả                    |
|---------------|-----------|-----------------|--------------------------|
| id            | id        | VARCHAR(255) PK | UUID tự sinh, khóa chính |
| fullName      | full_name | VARCHAR(255)    | Họ và tên đầy đủ         |
| email         | email     | VARCHAR(255)    | Email đăng               |

|             |                  |                             |                                |
|-------------|------------------|-----------------------------|--------------------------------|
|             |                  |                             | nhập, duy nhất                 |
| password    | password         | VARCHAR(255)                | Mật khẩu đã mã hóa BCrypt      |
| phoneNumber | phone_number     | VARCHAR(255)                | Số điện thoại (tùy chọn)       |
| dateOfBirth | date_of_birth    | DATE                        | Ngày sinh                      |
| gender      | gender           | ENUM(MALE,FEMALE,OTHER)     | Giới tính                      |
| avatarUrl   | avatar_url       | VARCHAR(255)                | URL ảnh đại diện (Supabase)    |
| verified    | verified         | BIT(1)                      | Email đã xác minh chưa         |
| roles       | user_roles.roles | ENUM(CUSTOMER,DRIVER,ADMIN) | Vai trò, bảng phụ user_roles   |
| createdAt   | created_at       | DATETIME(6)                 | Thời điểm tạo tài khoản (auto) |
| updatedAt   | updated_at       | DATETIME(6)                 | Thời điểm cập nhật cuối (auto) |

### Entity: RefreshToken – Bảng refresh\_token

Lưu Refresh Token đã hash (SHA-256) để hỗ trợ Refresh Token Rotation và thu hồi.

| Trường | Cột DB | Kiểu               | Mô tả        |
|--------|--------|--------------------|--------------|
| id     | id     | VARCHAR(255)<br>PK | UUID tự sinh |

|           |            |                        |                                    |
|-----------|------------|------------------------|------------------------------------|
| token     | token      | VARCHAR(255)<br>UNIQUE | SHA-256 hash của raw refresh token |
| user      | user_id    | FK → user.id           | Người dùng sở hữu token            |
| isRevoked | is_revoked | BIT(1)                 | Đã thu hồi hay chưa                |
| expiresAt | expires_at | DATETIME(6)            | Thời điểm hết hạn                  |
| createdAt | created_at | DATETIME(6)            | Thời điểm tạo (auto)               |

### Entity: BookingReview – Bảng booking\_review

Lưu đánh giá của hành khách cho một chuyến đi. Quan hệ 1-1 với Booking (mỗi booking chỉ được đánh giá một lần).

| Trường    | Cột DB      | Kiểu                      | Mô tả                       |
|-----------|-------------|---------------------------|-----------------------------|
| id        | id          | VARCHAR(255)<br>PK        | UUID tự sinh                |
| booking   | booking_id  | FK → booking.id<br>UNIQUE | Booking được đánh giá (1-1) |
| trip      | trip_id     | FK → trip.id              | Chuyến đi liên quan         |
| customer  | customer_id | FK → user.id              | Hành khách đánh giá         |
| driver    | driver_id   | FK → driver_profile.id    | Tài xế được đánh giá        |
| rating    | rating      | INT NOT NULL              | Số sao (1–5)                |
| comment   | comment     | TEXT                      | Nhận xét văn bản (tùy chọn) |
| createdAt | created_at  | DATETIME(6)               | Thời điểm tạo (auto)        |

## 3.2. Controller – API Endpoints

### AuthenticationController – /auth/\*\*

| Method | Endpoint       | Mô tả                        | Auth      |
|--------|----------------|------------------------------|-----------|
| POST   | /auth/register | Đăng ký tài khoản mới. Body: | Không cần |

|      |                              |                                                                |            |
|------|------------------------------|----------------------------------------------------------------|------------|
|      |                              | {fullName, email, password, role}                              |            |
| GET  | /auth/verification?token=... | Xác minh email qua token Redis                                 | Không cần  |
| POST | /auth/authentication         | Đăng nhập. Body: {email, password}. Trả về JWT + Refresh Token | Không cần  |
| POST | /auth/logout                 | Đăng xuất: xóa token Redis + revoke Refresh Token              | Bearer JWT |
| POST | /auth/refresh-token          | Làm mới Access Token bằng Refresh Token (Rotation)             | Không cần  |
| POST | /auth/request-otp            | Yêu cầu OTP để đổi mật khẩu (gửi email)                        | Bearer JWT |
| POST | /auth/change-password        | Đổi mật khẩu sau khi xác minh OTP                              | Bearer JWT |

### UserController – /users/\*\*

| Method | Endpoint         | Mô tả                                                                                     | Auth       |
|--------|------------------|-------------------------------------------------------------------------------------------|------------|
| GET    | /users/me        | Lấy thông tin người dùng đang đăng nhập                                                   | Bearer JWT |
| PUT    | /users/me        | Cập nhật thông tin cá nhân. Body: {fullName, phoneNumber, dateOfBirth, gender, avatarUrl} | Bearer JWT |
| PUT    | /users/me/avatar | Upload ảnh đại diện (multipart/form-data, field: file)                                    | Bearer JWT |

|     |                                  |                                  |            |
|-----|----------------------------------|----------------------------------|------------|
| GET | /users/me/quick/payment-summary  | Tóm tắt lịch sử thanh toán       | Bearer JWT |
| GET | /users/me/quick/security-summary | Kiểm tra mức độ hoàn thiện hồ sơ | Bearer JWT |

### CustomerBookingController – /customer/\*\* (liên quan đánh giá)

| Method | Endpoint                            | Mô tả                                                                                                         | Auth                  |
|--------|-------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------|
| POST   | /customer/bookings/{bookingId}/rate | Đánh giá chuyến đi.<br>Body: {rating: 1-5, comment: string}. Yêu cầu booking COMPLETED và chưa được đánh giá. | Bearer JWT (CUSTOMER) |

### 3.3. Service Layer – Các hàm nghiệp vụ chính

#### AuthenticationService

| Hàm                      | Mô tả                                                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| registerAccount(request) | Mã hóa mật khẩu BCrypt(10), gán role, lưu DB. Có comment-out block xác minh email bằng Redis token (có thể bật lại khi cần).                                                   |
| authenticate(request)    | Kiểm tra email + BCrypt password. Sinh JWT (HS512, claims: subject=userId, scope=ROLE_X). Lưu Redis: TOKEN:{jwt}=userId. Sinh UUID làm refresh token, hash SHA-256 rồi lưu DB. |
| logout(request)          | Xóa key Redis của access token. Revoke refresh token trong DB (isRevoked=true).                                                                                                |
| refreshToken(request)    | Xác minh refresh token (hash + DB lookup). Thu hồi token cũ. Sinh cặp token mới (Rotation). Lưu Redis token mới.                                                               |



|                         |                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------|
| requestOtp(request)     | Xác minh mật khẩu hiện tại, sinh OTP 6 chữ số, lưu Redis với TTL, gửi email qua MailService. |
| changePassword(request) | Kiểm tra OTP Redis. Mã hóa mật khẩu mới BCrypt. Revoke toàn bộ refresh token của user.       |
| generateToken(user)     | (Private) Tạo JWT HS512 với claims: subject, scope, issuer, expiration, jwtID (UUID).        |
| hashToken(token)        | (Private) Hash SHA-256 → Base64. Dùng cho cả Refresh Token và các token nhạy cảm.            |
| cleanupExpiredTokens()  | Cron job chạy 2AM hàng ngày, xóa refresh token hết hạn.                                      |

## UserService

| Hàm                     | Mô tả                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------|
| getCurrentUser()        | Lấy userId từ SecurityContextHolder, truy vấn DB. Dùng xuyên suốt các service khác.                |
| getMyInfo()             | Gọi getCurrentUser(), map sang UserResponse bằng Mapper.                                           |
| updateMyInfo(request)   | Cập nhật từng trường null-safe: fullName, phoneNumber, dateOfBirth, gender, avatarUrl. Trim chuỗi. |
| updateMyAvatar(request) | Upload file lên Supabase qua FileService.upload(), lưu path vào avatarUrl.                         |
| getMyPaymentSummary()   | Tổng hợp: tổng chuyến, chuyến hoàn thành, tổng chi, số đơn chưa thanh toán.                        |
| getMySecuritySummary()  | Đánh giá mức hoàn thiện hồ sơ (%), trả danh sách gợi ý bảo mật.                                    |

## CustomerBookingService (hàm đánh giá)

| Hàm                                     | Mô tả                                                                                                                                                  |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| createBookingReview(bookingId, request) | 1. Validate rating [1-5]. 2. Kiểm tra booking thuộc user hiện tại. 3. Kiểm tra trạng thái COMPLETED. 4. Kiểm tra chưa đánh giá (existsByBookingId). 5. |

Lưu BookingReview. 6. Cập nhật  
driver.driverRating = AVG từ DB.

### 3.4. Repository – Truy vấn CSDL

| Repository              | Phương thức đáng chú ý                                    | Mô tả                                                     |
|-------------------------|-----------------------------------------------------------|-----------------------------------------------------------|
| UserRepository          | findByEmail(email)                                        | Tìm user theo email, không lọc verified (dùng khi login)  |
| UserRepository          | findByEmailAndVerifiedIsTrue(email)                       | Tìm user đã xác minh email                                |
| RefreshTokenRepository  | findByTokenAndIsRevokedFalseAndExpiresAtAfter(token, now) | Tìm refresh token còn hiệu lực                            |
| RefreshTokenRepository  | revokeAllByUser(user)                                     | @Modifying: đặt isRevoked=true cho toàn bộ token của user |
| RefreshTokenRepository  | deleteExpiredTokens(now)                                  | @Modifying: xóa token hết hạn (cron job)                  |
| BookingReviewRepository | existsByBookingId(bookingId)                              | Kiểm tra booking đã có đánh giá chưa                      |
| BookingReviewRepository | averageRatingByDriverId(driverId)                         | @Query JPQL: tính AVG(rating                              |

|  |  |               |
|--|--|---------------|
|  |  | ) theo driver |
|--|--|---------------|

### 3.5. DTO – Cấu trúc Request/Response

| DTO Class                  | Loại     | Các trường                                                        | Dùng cho                 |
|----------------------------|----------|-------------------------------------------------------------------|--------------------------|
| AccountRegisterRequest     | Request  | fullName, email, password, role                                   | Đăng ký tài khoản        |
| AuthenticationRequest      | Request  | email, password                                                   | Đăng nhập                |
| LogoutRequest              | Request  | token, refreshToken                                               | Đăng xuất                |
| RefreshRequest             | Request  | refreshToken                                                      | Làm mới token            |
| SendOtpRequest             | Request  | password (xác minh)                                               | Yêu cầu OTP đổi mật khẩu |
| ChangePasswordRequest      | Request  | otp, newPassword                                                  | Đổi mật khẩu             |
| UpdateMyProfileRequest     | Request  | fullName, phoneNumber, dateOfBirth, gender, avatarUrl             | Cập nhật thông tin       |
| UpdateMyAvatarRequest      | Request  | file (MultipartFile)                                              | Upload ảnh đại diện      |
| CreateBookingReviewRequest | Request  | rating (Integer), comment (String)                                | Gửi đánh giá             |
| UserResponse               | Response | id, fullName, email, phoneNumber, gender, dateOfBirth, avatarUrl, | Trả về thông tin user    |

|                        |          |                                                         |                           |
|------------------------|----------|---------------------------------------------------------|---------------------------|
|                        |          | verified, roles, createdAt                              |                           |
| AuthenticationResponse | Response | token, refreshToken, authenticated, user (UserResponse) | Kết quả đăng nhập/refresh |
| BookingReviewResponse  | Response | id, bookingId, rating, comment, createdAt               | Kết quả đánh giá          |

### 3.6. Cấu hình bảo mật – SecurityConfig & CustomJwtDecoder

SecurityConfig cấu hình Spring Security theo chế độ Stateless (không session). Các endpoint /auth/\*\* được phép truy cập không cần xác thực, còn /users/\*\* và /api/\*\* yêu cầu Bearer JWT. Decoder tùy chỉnh CustomJwtDecoder thực hiện hai bước xác minh:

1. Xác minh chữ ký JWT bằng thuật toán HMAC-SHA512 với khóa bí mật từ application.properties.
2. Kiểm tra token còn tồn tại trong Redis (key: TOKEN:{jwt}). Nếu không có → token đã bị thu hồi (logout/expire) → từ chối truy cập ngay lập tức.

Cơ chế này đảm bảo rằng ngay cả JWT chưa hết hạn cũng bị vô hiệu hóa tức thì khi người dùng đăng xuất, giải quyết điểm yếu cố hữu của JWT stateless thuần túy.

### 3.7. API bên ngoài được sử dụng

| Dịch vụ                     | Mục đích                                                                                             | Tích hợp tại                                 |
|-----------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------|
| Supabase Storage            | Lưu trữ ảnh đại diện người dùng. File được upload qua HTTP REST API của Supabase, trả về public URL. | FileService.java, SupabaseStorageConfig.java |
| MailService (SMTP/SendGrid) | Gửi email xác minh tài khoản và email OTP đổi                                                        | MailService.java, AuthenticationService.java |

|                       |                                                      |                                                                     |
|-----------------------|------------------------------------------------------|---------------------------------------------------------------------|
|                       | mật khẩu. Cấu hình qua Spring Mail.                  |                                                                     |
| Redis (Upstash/local) | Cache Token, OTP, link xác minh. Hỗ trợ TTL tự động. | RedisConfig.java, AuthenticationService.java, CustomJwtDecoder.java |

## 4. Hướng Dẫn Cài Đặt và Triển Khai

### 4.1. Yêu cầu hệ thống

| Thành phần | Phiên bản tối thiểu | Ghi chú                                       |
|------------|---------------------|-----------------------------------------------|
| Java (JDK) | 17+                 | Bắt buộc cho Spring Boot 3.x                  |
| Maven      | 3.8+                | Quản lý dependency backend                    |
| MySQL      | 8.0+                | Database chính                                |
| Redis      | 6.0+                | Cache token, OTP. Có thể dùng Upstash (cloud) |
| Node.js    | 18+                 | Chạy frontend React Native (Expo)             |
| Expo CLI   | Mới nhất            | npm install -g expo-cli                       |

### 4.2. Cài đặt Backend (Spring Boot)

- Clone repository: `git clone https://github.com/hungnx-yb/Ride_Up.git`
- Tạo database MySQL: `CREATE DATABASE rideUp;`
- Import dữ liệu mẫu: `mysql -u root -p rideUp < dump-rideUp-202603180851.sql`
- Cấu hình file `application.properties` (hoặc `application.yml`):

```
spring.datasource.url=jdbc:mysql://localhost:3306/rideUp
spring.datasource.username=root
spring.datasource.password=YOUR_PASSWORD
spring.jpa.hibernate.ddl-auto=update
```

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

jwt.signerKey=YOUR\_SECRET\_KEY\_MIN\_64\_CHARS  
jwt.valid-duration=3600 # 1 giờ  
jwt.refreshable-duration=2592000 # 30 ngày

supabase.url=https://xxx.supabase.co  
supabase.key=YOUR\_SUPABASE\_SERVICE\_KEY

spring.mail.host=smtp.gmail.com  
spring.mail.port=587  
spring.mail.username=YOUR\_EMAIL  
spring.mail.password=YOUR\_APP\_PASSWORD

7. Chạy backend: cd RideUp && ./mvnw spring-boot:run
8. Backend khởi động tại: http://localhost:8080

#### 4.3. Cài đặt Frontend (React Native / Expo)

9. Vào thư mục frontend: cd RideUpUI
10. Cài dependencies: npm install
11. Cấu hình API URL trong src/config/config.js (trỏ đến IP backend)
12. Chạy ứng dụng: npx expo start
13. Quét QR code bằng Expo Go (iOS/Android) hoặc nhấn 'w' để chạy Web

#### 4.4. Lưu ý khi triển khai

- JWT signerKey: Độ dài tối thiểu 64 ký tự cho HMAC-SHA512. Không commit lên Git. Dùng biến môi trường.
- Redis bắt buộc: Backend sẽ báo lỗi nếu không kết nối được Redis vì mọi xác thực đều kiểm tra Redis.
- CORS: SecurityConfig đang cho phép tất cả origin (allowedOriginPattern: \*). Cần restrict lại khi deploy production.
- Email xác minh: Hiện tại bị TẮT trong code (user.setVerified(true) mặc định). Để bật lại, uncomment block email trong AuthenticationService.registerAccount().
- booking\_review table: Bảng này chưa có trong file dump SQL do được tạo tự động qua JPA (ddl-auto=update). Đảm bảo ddl-auto=update hoặc create khi chạy lần đầu.
- File upload size: Mặc định Spring Boot giới hạn 1MB. Cần cấu hình spring.servlet.multipart.max-file-size=10MB nếu muốn upload ảnh lớn.
- Cron job cleanup: AuthenticationService có @Scheduled chạy lúc 2AM mỗi ngày để dọn refresh token hết hạn. Cần bật @EnableScheduling trong main class nếu chưa có.

## PHẦN II: MÔ TẢ CODE VÀ LINK GITHUB

### 1. Link Repository

GitHub: <https://github.com/tuanhy1702/RideUp>

### 2. Cấu Trúc Thư Mục

| Đường dẫn                                                                       | Mô tả                                                                          |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| RideUp/src/main/java/com/example/demo/controller/AuthenticationController.java  | API đăng ký, đăng nhập, logout, refresh token, OTP, đổi mật khẩu               |
| RideUp/src/main/java/com/example/demo/controller/UserController.java            | API lấy/cập nhật thông tin user, upload avatar                                 |
| RideUp/src/main/java/com/example/demo/controller/CustomerBookingController.java | API đặt xe, hủy, đánh giá chuyên đi (method: createBookingReview tại dòng 147) |
| RideUp/src/main/java/com/example/demo/service/AuthenticationService.java        | Logic đăng ký, xác thực, JWT, refresh token rotation, OTP                      |
| RideUp/src/main/java/com/example/demo/service/UserService.java                  | Logic cập nhật thông tin, upload avatar, tóm tắt tài khoản                     |
| RideUp/src/main/java/com/example/demo/service/CustomerBookingService.java       | Logic đặt xe, đánh giá (createBookingReview tại dòng 485)                      |
| RideUp/src/main/java/com/example/demo/entity/User.java                          | Entity bảng user                                                               |

|                                                                               |                                                          |
|-------------------------------------------------------------------------------|----------------------------------------------------------|
| RideUp/src/main/java/com/example/demo/entity/RefreshToken.java                | Entity bảng refresh_token                                |
| RideUp/src/main/java/com/example/demo/entity/BookingReview.java               | Entity bảng booking_review                               |
| RideUp/src/main/java/com/example/demo/repository/UserRepository.java          | JPA Repository cho User                                  |
| RideUp/src/main/java/com/example/demo/repository/RefreshTokenRepository.java  | JPA Repository cho RefreshToken (có @Modifying queries)  |
| RideUp/src/main/java/com/example/demo/repository/BookingReviewRepository.java | JPA Repository cho BookingReview (có AVG query)          |
| RideUp/src/main/java/com/example/demo/config/SecurityConfig.java              | Cấu hình Spring Security, CORS, OAuth2 Resource Server   |
| RideUp/src/main/java/com/example/demo/config/CustomJwtDecoder.java            | JWT decoder tùy chỉnh, kiểm tra Redis (token revocation) |
| RideUp/src/main/java/com/example/demo/dto/request/*.java                      | Tất cả DTO request liên quan đến auth, user, review      |
| RideUp/src/main/java/com/example/demo/dto/response/*.java                     | Tất cả DTO response liên quan đến auth, user, review     |
| RideUpUI/src/screens/auth/LoginScreen.js                                      | Màn hình đăng nhập React Native                          |
| RideUpUI/src/screens/auth/RegisterScreen.js                                   | Màn hình đăng ký React Native                            |



|                              |                                                                                                    |
|------------------------------|----------------------------------------------------------------------------------------------------|
| RideUpUI/src/services/api.js | Service gọi API:<br>login(),<br>register(),<br>updateMyProfile()<br>updateMyAvatar()<br>rateRide() |
| dump-rideUp-202603180851.sql | Dump database MySQL (bảng user, refresh_token, booking, ...)                                       |

### 3. Mô Tả Comment trong Code

Theo yêu cầu báo cáo, các file liên quan đến phần cá nhân thực hiện đã được comment đầy đủ. Dưới đây là tóm tắt vị trí comment quan trọng:

| File                        | Vị trí comment đáng chú ý                                                                                                                                  |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AuthenticationService.java  | Block TODO comment về bật/tắt xác minh email (dòng ~52-70). Chú thích từng bước trong registerAccount(), authenticate(), refreshToken(), changePassword(). |
| UserController.java         | Javadoc trên method getMyInfo() giải thích yêu cầu Bearer token.                                                                                           |
| UserService.java            | Comment null-safe pattern trong updateMyInfo(). Giải thích logic normalize() và normalizeNullable().                                                       |
| CustomJwtDecoder.java       | Comment double-checked locking pattern cho lazy init NimbusJwtDecoder. Giải thích kiểm tra Redis.                                                          |
| RefreshTokenRepository.java | Comment inline trên deleteExpiredTokens() giải thích mục đích cleanup job.                                                                                 |
| api.js (Frontend)           | Comment /** Đăng xuất */ và mô tả từng API function: login, register, updateMyProfile, rateRide.                                                           |

